

Diagnóstico do Sistema de Empréstimo

Problema 1: Validação de Prazo Mínimo de Empréstimo

O que a documentação diz: A RN02 em requisitos.md estabelece que o prazo mínimo de empréstimo é de 1 dia.

O que o código faz: O método registrar() em emprestimos.py calcula a data de devolução sem verificar se a quantidade de dias informada é maior que zero, aceitando valores inválidos como 0 ou negativos.

Por que é um problema: Permite a criação de empréstimos com prazos incoerentes, que podem resultar em datas de devolução no passado ou empréstimos instantâneos, gerando inconsistências no sistema.

(Já documentado? não)

Problema 2: Confirmação Explícita de Operação

O que a documentação diz: A RI02 em requisitos.md exige que toda operação bem-sucedida exiba uma mensagem de confirmação explícita ao usuário.

O que o código faz: Após o registro de um empréstimo, o método registrar() apenas exibe uma simulação de e-mail ([EMAIL]...), sem uma mensagem clara e direta de que o empréstimo foi registrado com sucesso.

Por que é um problema: A falta de feedback explícito pode confundir o usuário sobre o sucesso da operação, levando a dúvidas ou tentativas repetidas de registro.

(Já documentado? não)

Problema 3: Tratamento de Opção de Menu Inválida

O que a documentação diz: A RI03 em requisitos.md especifica que toda operação inválida deve exibir uma mensagem de erro descritiva sem encerrar o sistema.

O que o código faz: Na função main(), ao digitar uma opção de menu que não seja 0, 1, 2 ou 3, o sistema simplesmente repete o menu sem exibir qualquer mensagem de erro ou feedback ao usuário.

Por que é um problema: A ausência de uma mensagem de erro descritiva para opções inválidas torna a interface pouco amigável e dificulta para o usuário entender o que deu errado e como proceder.

(Já documentado? não)

Problema 4: Mensagem para Empréstimos em Atraso Vazios

O que a documentação diz: O UC03, passo 4, em casos de uso.md indica que, se não houver empréstimos em atraso, o sistema deve exibir a mensagem "Nenhum empréstimo em atraso".

O que o código faz: O método listar_atrasados() em emprestimos.py não imprime nenhuma mensagem quando a lista de empréstimos em atraso está vazia, resultando em uma saída vazia para o usuário.

Por que é um problema: A falta de uma mensagem clara quando não há atrasos pode levar o usuário a acreditar que houve um erro ou que a função não está operando corretamente, em vez de indicar a ausência de resultados.

(Já documentado? não)

Problema 5: Cálculo de Multa Duplicado

O que a documentação diz: A DT04 na Tabela de Dívida Técnica em projeto.md aponta que o cálculo de multa está duplicado em dois métodos.

O que o código faz: O bloco de código responsável pelo cálculo da multa com base no tipo de equipamento é repetido nos métodos devolver() e listar_atrasados() em emprestimos.py.

Por que é um problema: A duplicação de código viola o princípio DRY (Don't Repeat Yourself), tornando a manutenção mais difícil e propensa a erros, pois qualquer alteração na lógica de cálculo da multa precisaria ser feita em múltiplos locais.

(Já documentado? sim)

Problema 6: Mistura de Notificação com Lógica de Negócio

O que a documentação diz: A DT03 na Tabela de Dívida Técnica em projeto.md identifica a notificação misturada com a lógica de negócio como um problema.

O que o código faz: As chamadas para `print(f"[EMAIL]...")` que simulam o envio de e-mails estão diretamente inseridas nos métodos `registrar()`, `devolver()` e `listar_atrasados()` em `emprestimos.py`, misturando responsabilidades de comunicação com a lógica central do sistema.

Por que é um problema: Essa mistura de responsabilidades dificulta a manutenção, a testabilidade e a reutilização do código, além de tornar o sistema menos flexível para futuras mudanças na forma como as notificações são tratadas.

(Já documentado? sim)

Problema 7: Uso de Variáveis Globais e Dicionários em vez de Classes de Domínio

O que a documentação diz: A DT02 e a observação do desenvolvedor na seção 3 de `projeto.md` mencionam o uso de variáveis globais e dicionários em vez de classes de domínio como uma dívida técnica intencional.

O que o código faz: O sistema utiliza listas de dicionários (`equipamentos` e `emprestimos_registrados`) como variáveis globais para armazenar os dados, e a classe `Sistema` acessa e manipula diretamente essas estruturas globais.

Por que é um problema: O uso de variáveis globais e dicionários sem tipagem explícita reduz a clareza do código, dificulta a validação automática de dados, e aumenta o acoplamento entre as partes do sistema, comprometendo a manutenibilidade e a testabilidade.

(Já documentado? sim)

Problema 8: Violação do Princípio OCP no Cálculo de Multa

O que a documentação diz: A DP05 e a DT06 em `projeto.md` indicam que o cálculo de multa com `if/elif` por tipo viola o Princípio do Aberto/Fechado (OCP).

O que o código faz: Os métodos `devolver()` e `listar_atrasados()` contêm uma estrutura `if/elif` para determinar o valor da multa com base no `tipo` do equipamento.

Por que é um problema: Para adicionar um novo tipo de equipamento, seria necessário modificar diretamente esses métodos, o que viola o OCP. Isso torna o sistema menos extensível e mais propenso a erros ao introduzir novas funcionalidades.

(Já documentado? sim)

Problema 9: Zero Testes Automatizados

O que a documentação diz: A DP04 e a DT05 em [projeto.md](#) afirmam que não há testes automatizados, e a RNF04 em [requisitos.md](#) indica que a testabilidade não foi atendida.

O que o código faz: Não há nenhum arquivo de teste ou estrutura de testes automatizados presente no repositório, e o código não foi projetado para ser facilmente testável de forma isolada.

Por que é um problema: A ausência de testes automatizados aumenta significativamente o risco de regressões ao fazer alterações, dificulta a refatoração e a verificação da correção do sistema, comprometendo a qualidade e a confiabilidade do software.

(Já documentado? sim)